

Timeflow Test Automation Framework

Comprehensive Technical & Architectural Engineering Report

Target Application:	https://timeflow.setoo.in	Build / Suite Status:	52 Passed / 0 Failed (100%)
Primary Language:	Java 17 (LTS)	Automation Core:	Selenium WebDriver 4.29.0
Test Runner:	TestNG 7.10.2	Design Pattern:	Page Object Model (POM)
Git Repository:	github.com/darshanhujband8055/timeflow-selenium-automation	ALM / Project:	Azure DevOps (setoo-ai / Timeflow)

1. Executive Summary

The Timeflow Selenium Automation Framework is an enterprise-grade quality assurance solution designed to validate end-to-end user workflows, access security, and business invariants across all 4 user roles on the Setoo Timeflow application. The framework was engineered with a primary focus on **zero-friction portability**: any developer, QA, or manager can clone the repository and run all 52 tests locally via a 1-click batch runner without manually configuring build runtimes or browser binaries.

2. Tools, Technologies & Services Stack

Category	Technology / Tool	Version / Role	Technical Significance
Language	Java	17 (LTS)	Provides strong static typing, OOP reliability, and enterprise stability.
Core Automation	Selenium WebDriver	4.29.0	Automates Chrome, Edge, and Firefox. Bundled Selenium Manager manages driver binaries automatically.
Test Runner	TestNG	7.10.2	Drives test suite orchestration, lifecycle hooks, assertions, and test listeners.
Build Automation	Apache Maven	3.9+ & mvnw	Build lifecycle, dependency management, and Surefire execution reporting.
Target Frontend	Next.js / React	SPA / Radix UI	Modern web app with async hydration, interactive tab navigation, and client modals.
Target Backend	PostgreSQL / Node.js	Supabase / REST	Relational data store persisting timesheets, project budgets, and user entitlements.
Authentication	JWT / OAuth	Session Guard	Role-gated sessions across Employee, Manager, PM, and Administrator.
Version Control	Git & GitHub	main branch	Centralized repository with automated wrapper scripts and documentation.
ALM Integration	Azure DevOps	setoo-ai	Connected to project 'Timeflow timesheet' for work item alignment and bug tracking.

3. Framework Architecture & Design Pattern

The framework strictly follows the industry-standard **Page Object Model (POM)** architecture, cleanly separating element locators, user actions, and test verification logic across discrete modular layers:

- **Page Object Layer:** Encapsulates page elements (`private final By ...`) and reusable interaction flows within dedicated classes (e.g. `EmployeeTimesheetsPage`, `PMBillingPage`, `ManagerApprovalsPage`). Locators are isolated so UI changes require updates in only one file.
- **ThreadLocal Driver Management (`DriverManager.java`):** Wraps `WebDriver` inside a `ThreadLocal` container to ensure thread safety, complete test isolation, and seamless multi-threaded parallel test capability.
- **Dynamic Synchronization (`BasePage.java`):** Replaces brittle hardcoded pauses with intelligent explicit synchronization using `WebDriverWait` and `ExpectedConditions` (`elementToBeClickable`, `visibilityOf`, `invisibilityOfElementLocated`).
- **Centralized Configuration (`config.properties`):** Allows dynamic runtime toggling of target browser (chrome/edge/firefox), application URL, headless mode (`-Dheadless=true`), and timeout thresholds without modifying source code.

4. Execution Workflow & Zero-Setup Portability

To enable managers and non-technical stakeholders to easily run and review tests, the framework is bundled with multiple zero-setup execution mechanisms:

Execution Method	Command / Action	Target Audience & Workflow
1-Click Windows Runner	Double-click run-all-tests.bat	Ideal for managers. Bundled Maven Wrapper handles setup automatically and launches the full suite in Google Chrome.
1-Click Mac/Linux Runner	./run-all-tests.sh	Shell script for macOS and Linux workstations with execution permissions pre-configured.
Maven CLI (Full Suite)	./mvnw clean test	Executes all 52 tests mapped in testng.xml across all 4 business modules.
Headless Mode (CI/CD)	./mvnw test -Dheadless=true	Runs silently in the background without UI popups—ready for Azure DevOps CI pipelines.
Module-Specific Run	./mvnw test -Dtest=ManagerModuleTest	Selectively executes only the tests for a target module or role.
IDE Execution	Right-click testng.xml -> Run	For developers and SDETs working in IntelliJ IDEA, VS Code, or Eclipse.

5. Failure Handling & Diagnostic Strategy

Robust automated diagnostics ensure that any regression is immediately actionable for developers:

- **Automated Failure Screenshots:** Integrated via TestNG's `ITestListener` (`TestListener.java`). Any unexpected assertion or timeout failure automatically captures a full-screen timestamped PNG screenshot saved directly to `target/screenshots/`.
- **Surefire HTML Reports:** Generated at `target/surefire-reports/index.html` detailing test class run times, method parameters, failure traces, and aggregate pass rates.
- **Driver Auto-Cleanup:** The `@AfterMethod` lifecycle hook guarantees that browser windows and driver instances terminate cleanly even if assertions fail, preventing orphaned processes from leaking machine memory.

6. Module-by-Module Automation Script Breakdown

Module A: Standard Employee Role (shubham.shinde@setoo.co)

Test Classes: EmployeeModuleTest.java, EmployeeTimesheetWorkflowTest.java, EmployeeAuthenticationTest.java, EmployeePermissionsTest.java

Scope: Daily, Weekly, and Monthly timesheet views; 'Sync My Tasks' project integration; hours logging across task rows; dynamic weekly total calculation; 'Save draft' persistence; 'Submit week' modal guardrails; navigation to My Tasks, My Allocation, and Personal Calendar; negative security tests ensuring admin/billing routes are forbidden.

Module B: Reporting Manager Role (rutuja@setoo.co)

Test Classes: ManagerModuleTest.java

Scope: Approvals queue processing; table checkbox multi-select; batch approval workflows; batch rejection workflows; rejection feedback comment enforcement (DEF-MGR-001 audit); Team Directory member profiles; Bandwidth allocation gauges; capacity utilization thresholds; custom date range filtering and CSV report exports.

Module C: Project Manager Role (rohan@setoo.co)

Test Classes: PMModuleTest.java

Scope: Project portfolio overview; project health status badges; sprint velocity metrics; Billing & Budget dashboard (/billing); financial stat cards (Planned vs. Consumed hours, Revenue); Extra Burned calculation verification (DEF-PM-001 audit); mutation authorization guardrails (TC 1.3 - PMs restricted from altering financial baselines).

Module D: Administrator Role (darshan@setoo.co)

Test Classes: AdminModuleTest.java, AdminRejectionWorkflowTest.java, AccessControlAndPermissionsTest.java

Scope: Full master privilege verification; organizational overview cards (Active Users, Active Projects, ADO Mapped status, Pending Approvals); tab navigation (Overview, Approvals Analytics, Pending Submissions); Organization Settings (/settings); global security RBAC negative tests ensuring lower roles cannot access admin routes.

7. Defect Audits & Next Steps

A total of **17 functional defects** across all 4 modules were identified during framework execution and exploratory testing. Each defect has been cataloged with reproduction steps, root cause analysis, and developer remediation code in the module PDF reports. The complete test framework is available on GitHub at

<https://github.com/darshanhujband8055/timeflow-selenium-automation>.